Name: ______________________________________________ Class number: _____
Section: __________Schedule:_____________________________ Date: ____________

**Lesson Title:** Advance GUI
**Lesson Targets:**
At the end of this module, students will be able to:
1. Know the use of layout managers and panels.
2. Create and apply different layout managers and panels into programs.
3. Create advanced event-driven programs and other mouse-event activities.

**Materials:**
SAS
**References:**
- Farrell,Joyce.*Java Programming: Concepts and Applications*. Second Edition Cengage Learning Asia 2012.
- *https://github.com/java-tutorial-for-beginners*

## A. LESSON PREVIEW/REVIEW

### Introduction

Good day to everyone. In this module you will learn about different layouts which will help you in building an oriented graphical user interface to your program using swing and awt components.

## B. MAIN LESSON

### Layout Managers

A layout manager is an object that controls the size and position (that is, the layout) of components inside a Container object. The layout manager that you assign to the window determines how the components are sized and positioned within the window.

| Layout Manager | When to Use |
|---|---|
| BorderLayout | Use when you add components to a maximum of five sections arranged in north, south, east, west, and center positions. |
| FlowLayout | Use when you need to add components from left to right; FlowLayout automatically moves to the next row when needed, and each component takes its preferred size. |
| GridLayout | Use when you need to add components into a grid of rows and columns; each component is the same size. |
| CardLayout | Use when you need to add components that are displayed one at a time. |
| BoxLayout | Use when you need to add components into a single row or a single column. |
| GridBagLayout | Use when you need to set size, placement, and alignment constraints for every component that you add. |

**Border Layout Manager-** is the default manager class for all content panes. You can use the BorderLayout class with any container that has five or fewer components.

When you use the BorderLayout manager, the components fill the screen in five regions: north, south, east, west, and center.

Name: ____________________________________________ Class number: _____
Section: ___________Schedule:______________________________ Date: ___________

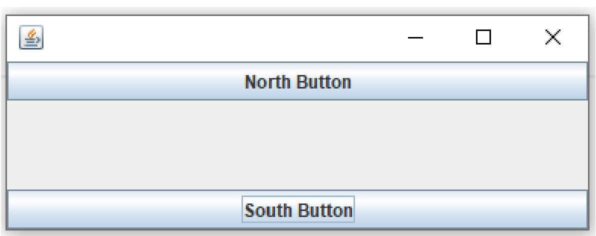
When you add a component to a container that uses BorderLayout, the **add()** method uses two arguments: the component and the region to which the component is added. The BorderLayout class provides five named constants for the regions—**BorderLayout.NORTH**,. **SOUTH**,. **EAST**, .**WEST**, and .**CENTER**.

Example:

```
private JButton nb = new JButton("North Button");
private JButton sb = new JButton("South Button");

setLayout(new BorderLayout());
add(nb, BorderLayout.NORTH);
add(sb, BorderLayout.SOUTH);
```

Output:




**FlowLayout Manager - class** to arrange components in rows across the width of a Container—you used FlowLayout with the content panes of JFrames. With FlowLayout, each Component that you add is *placed to the* ***right*** of previously added components in a row; or, if the current row is filled, the Component is placed to start a new row.

The **FlowLayoutclass** contains three constants you can use to align Components with a Container:

a) FlowLayout.LEFT
b) FlowLayout.CENTER
c) FlowLayout.RIGHT

Example

```
private JButton lb = new JButton("L Button");
private JButton rb = new JButton("R Button");
private Container con = getContentPane();
//inside the FlowLayout() you can define default orientation of the components, by default
//it will be placed on the center
private FlowLayout layout = new FlowLayout();
con.setLayout(layout)
```

Name: ______________________________________________________ Class number: _____
Section: ___________Schedule:_________________________________ Date: ____________

Output:


**GridLayout Manager**- use to arrange components into equal rows and columns. When you create a GridLayout object, you indicate the numbers of rows and columns you want, and then the container surface is divided into a grid, much like the screen you see when using a spreadsheet program.

Syntax:

```
con.setLayout(new GridLayout (4, 5));
```

the following statement establishes an anonymous GridLayout with four horizontal rows and five vertical columns in a Container named con:

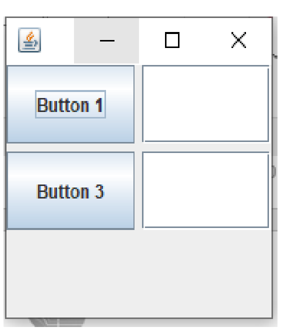
Similarly, within a JFrame, you can establish the same layout with either of the following:

```
this.setLayout(new GridLayout(4, 5));
setLayout(new GridLayout(4, 5));
```

**Example**

```
private JButton b1 = new JButton("Button 1");
private JTextField t1 = new JTextField(10);
private JButton b3 = new JButton("Button 3");
private JTextField t2 = new JTextField(10);
    // 3 rows, 2 columns, 5horizontal gap, 5 vertical gap
private GridLayout layout = new GridLayout(3, 2, 5, 5);
setLayout(layout);
 add(b1);
 add(t1);
 add(b3);
  add(t2);
```

Output:



Name: ______________________________________________ Class number: _____
Section: __________Schedule:______________________________ Date: __________

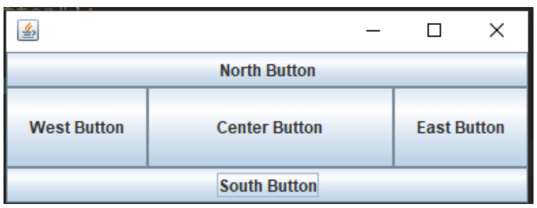
Note*: You may refer to the **Day 18** video resources in demonstrating an event that has something to do with key Event and mouseEvent.

**Skill-building Activities**

You will need to come up with the proper solution to the following exercise:

Exercise 1: Create a Container that represents the components from a designated location

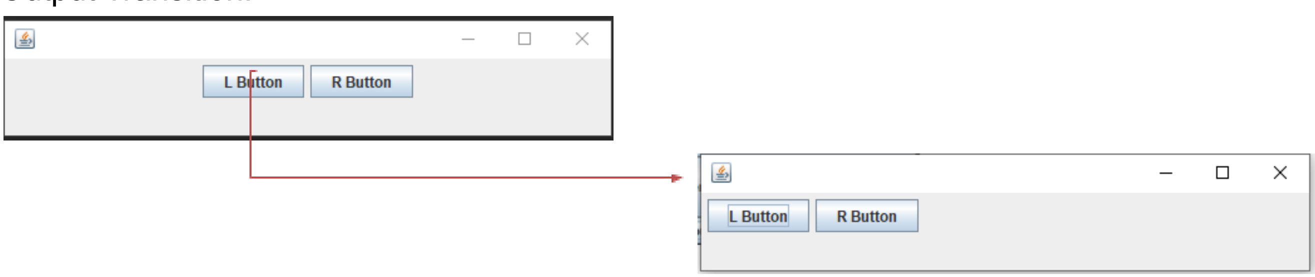
Output:




Exercise 2: Create Container that contains two (2) button components and assign a function:

a. Label with "Left Button" and "Right Button"
b. When the "left button" is click it brings the two components to the left corner
c. Similarly, with the "right" button it reverses the directions of the component

Output Transition:




**Check for Understanding**

Identify what is being asked on the following questions

____________1. It overrides the default behavior of JFrame to be positioned in different directions
____________2. A component which a user can type a single line of text
____________3. It sets the size of JFrame using dimensions object
____________4. A method that triggers an event in particular when using JButton.
____________5. Visual experience builder for Java applications

Name: ______________________________________________________ Class number: _____
Section: ___________Schedule:_________________________________ Date: ____________

**C. LESSON WRAP-UP**

**FAQs**

**a)** Not all Events are ActionEvents with an addActionListener()method. For example, KeyListeners have an addKeyListener()method, and FocusListeners have an addFocusListener()method.

**Thinking about Learning**

**b)** Mark your place in the work tracker which is simply a visual to help you track how much work you have accomplished and how much work there is left to do.

| Period 1 | | | | | | | | Period 2 | | | | | | | Period 3 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 |

**c)** Think about your learning by filling up “My Learning Tracker”. Write the learning targets, score, and learning experience for the session and deliberately plan for the next session.

| Date | Learning Target/Topic | Scores | Action Panel |
|---|---|---|---|
| What’s the date today? | What module # did you do? What were the learning targets? What activities did you do? | What were your scores in the activities? | What contributed to the quality of your performance today? What will you do next session to maintain your performance or improve it? |
| | | | |